Hands-on Activity <8.1>	
<Inheritance, Encapsulation, and Abstraction>	
Course Code: CPE009A	**Program:** Computer Engineering
Course Title: Object Oriented Programming 1	**Date Performed:03/18/2026**
Section: CPE12S1	**Date Submitted:03/18/2026**
Name(s):Will Stuart D. Ponce Jr.	**Instructor:** Engr. Jimlord M. Quejado

6. Output

Output code:

```
class Character():
    def __init__(self, username):
        self._username = username # [cite: 86-87]
        self._hp = 100 # [cite: 89]
        self._mana = 100 # [cite: 92]
        self._damage = 5 # [cite: 95]
        self._str = 0 # [cite: 98]
        self._vit = 0 # [cite: 100]
        self._int = 0 # [cite: 102]
        self._agi = 0 # [cite: 114]

    def getUsername(self):
        return self._username # [cite: 115]

    def setUsername(self, new_username):
        self._username = new_username # [cite: 117]

    def getHp(self):
        return self._hp # [cite: 112]

    def setHp(self, new_hp):
        self._hp = new_hp # [cite: 120]

    def getDamage(self):
        return self._damage # [cite: 133]

    def getStr(self):
        return self._str # [cite: 137]
```

```
    def setStr(self, new_str):
        self._str = new_str # [cite: 139]

    def getVit(self):
        return self._vit # [cite: 141]

    def setVit(self, new_vit):
        self._vit = new_vit # [cite: 144]

    def getInt(self):
        return self._int # [cite: 149]

    def setInt(self, new_int):
        self._int = new_int # [cite: 152]

    def getAgi(self):
        return self._agi # [cite: 156]

    def setAgi(self, new_agi):
        self._agi = new_agi # [cite: 165]

    def reduceHp(self, damage_amount):
        self._hp = self._hp - damage_amount # [cite: 167]

    def addHp(self, heal_amount):
        self._hp = self._hp + heal_amount # [cite: 168]
```

Code:

```
>>> print("Hello World")
Hello World
>>> class Character():
...     def __init__(self, username):
...         self._username = username # [cite: 86-87]
...         self._hp = 100 # [cite: 89]
...         self._mana = 100 # [cite: 92]
...         self._damage = 5 # [cite: 95]
...         self._str = 0 # [cite: 98]
...         self._vit = 0 # [cite: 100]
...         self._int = 0 # [cite: 102]
...         self._agi = 0 # [cite: 114]
...     def getUsername(self):
...         return self._username # [cite: 115]
...     def setUsername(self, new_username):
...         self._username = new_username # [cite: 117]
...     def getHp(self):
...         return self._hp # [cite: 112]
...     def setHp(self, new_hp):
...         self._hp = new_hp # [cite: 120]
```

```
...         self._str = new_str # [cite: 139]
...     def getVit(self):
...         return self._vit # [cite: 141]
...     def setVit(self, new_vit):
...         self._vit = new_vit # [cite: 144]
...     def getInt(self):
...         return self._int # [cite: 149]
...     def setInt(self, new_int):
...         self._int = new_int # [cite: 152]
...     def getAgi(self):
...         return self._agi # [cite: 156]
...     def setAgi(self, new_agi):
...         self._agi = new_agi # [cite: 165]
...     def reduceHp(self, damage_amount):
...         self._hp = self._hp - damage_amount # [cite: 167]
...     def addHp(self, heal_amount):
...         self._hp = self._hp + heal_amount
```

Code Analysis:

The Character class acts as an efficient base class in the Object-Oriented Programming (OOP) model, focusing on the Encapsulation concept through the use of an underscore prefix (e.g., _username, _hp) to indicate that these variables must be kept hidden from external access . However, the Character class offers an extensive list of Getter and Setter methods that must be followed in order to access the variables, such as the getUsername() and setHp() methods . The __init__ constructor initializes the newly created character with standard default stats, including 100 HP, 100 Mana, and 5 damage . The implementation of particular RPG-style stats, such as str, vit, int, and agi, lays the groundwork for the concept of Inheritance, as these stats may be adjusted in derived classes to create distinct classes such as Swordsmen or Magicians . The Character class's implementation of methods such as reduceHp() and addHp() effectively hides the arithmetic behind the game's logic, keeping the game loop clean and simple.

Output code:

```
from Character import Character

class Novice(Character):
    def basicAttack(self, character):
        character.reduceHp(self.getDamage())
        print(f"{self.getUsername()} performed Basic Attack! -{self.getDamage()}")
```

Code:

```
>>> class Boss(Swordsman, Archer, Magician):  # multiple inheritance
...     def __init__(self, username):
...         super().__init__(username)
...         self.setStr(10)
...         self.setVit(25)
...         self.setInt(5)
...         self.setHp(self.getHp() + self.getVit())
...
```

Code Analysis:

Your RPG system shows good understanding of Object-oriented programming (OOP) concepts, as evidenced by the modular class structure of your design, utilizing multiple inheritance of the Boss class, allowing it to inherit the properties of the Swordsman, Archer, and Magician classes, creating a composite class of these three archetypes. The implementation of the getter and setter methods, such as setStr() and getHp(), shows good understanding of the encapsulation principle of OOP, creating a controlled environment of attribute modification. The basicAttack method shows good implementation of inter-object communication, as evidenced by the reduction of the target character's health points by calling the target character's reduceHp method.

The yellow squiggly line in your IDE, however, shows that the module resolution of the import Character statement is not recognized, and this needs to be resolved accordingly. All parent classes should utilize the super().__init__() method when utilizing multiple inheritance, as this helps traverse the Method Resolution Order of the class without skipping the initialization of crucial attributes.

Output Code:

```
from Novice import Novice

class Swordsman(Novice):
    # Fixed: Changed _init_ to __init__
    def __init__(self, username):
        # Fixed: Changed _init__ to __init__
        super().__init__(username)
        self.setStr(5)
        self.setVit(10)
        self.setHp(self.getHp() + self.getVit())

    def slashAttack(self, character):
        self.new_damage = self.getDamage() + self.getStr()
        character.reduceHp(self.new_damage)
        # Fixed: Added missing curly braces for the username
        print(f"{self.getUsername()} performed Slash Attack! -{self.new_damage}")
```

Code:

```
>>> class Boss(Swordsman, Archer, Magician):  # multiple inheritance
...     def __init__(self, username):
...         super().__init__(username)
...         self.setStr(10)
...         self.setVit(25)
...         self.setInt(5)
...         self.setHp(self.getHp() + self.getVit())
```

Code Analysis:

The code provided is an example of multiple inheritance and encapsulation in object-oriented programming in a game-like scenario. In the provided script, the Swordsman class inherits properties from a Novice class, which is properly utilized with super().__init__(username) in initializing inherited properties and overriding base properties such as strength and vitality with setter methods. In addition, the slashAttack method is an example of interaction between objects, where the amount of damage is dynamically calculated based on properties and the state change of another character. In the terminal, the Boss class extends multiple classes: Swordsman, Archer, and Magician. This is an example of multiple inheritance, where the Boss class is able to use properties and methods of all three classes. However, there is complexity in using multiple inheritance in Python, as it uses a specific sequence in which methods are inherited using super(), which may not be desirable in all cases.

Output Code:

Archer.py > Archer > rangedAttack

```python
from Novice import Novice
import random

class Archer(Novice):
    # Fixed: Changed _init_ to __init__
    def __init__(self, username):
        super().__init__(username)
        self.setAgi(5)
        self.setInt(5)
        self.setVit(5)
        self.setHp(self.getHp() + self.getVit())

    def rangedAttack(self, character):
        self.new_damage = self.getDamage() + random.randint(0, self.getAgi())
        character.reduceHp(self.new_damage)
        # Fixed: Changed (...) to {...} for f-string variables
        print(f"{self.getUsername()} performed Ranged Attack! -{self.new_damage}")
```

Code:

```python
>>> class Boss(Swordsman, Archer, Magician):  # multiple inheritance
...     def __init__(self, username):
...         super().__init__(username)
...         self.setStr(10)
...         self.setVit(25)
...         self.setInt(5)
...         self.setHp(self.getHp() + self.getVit())
...
```

Code Analysis:

Output Code:

```python
from Novice import Novice

class Magician(Novice):
    def __init__(self, username):
        super().__init__(username) [cite: 257]
        self.setInt(10) [cite: 258]
        self.setVit(5) [cite: 259]
        self.setHp(self.getHp() + self.getVit()) [cite: 260]

    def heal(self):
        self.addHp(self.getInt()) [cite: 262]
        print(f"{self.getUsername()} performed Heal! +{self.getInt()}") [cite: 263]

    def magicAttack(self, character):
        self.new_damage = self.getDamage() + self.getInt() [cite: 265]
        character.reduceHp(self.new_damage) [cite: 266]
        print(f"{self.getUsername()} performed Magic Attack! -{self.new_damage}") [cite: 267]
```

Code:

```
>>> class Magician(Novice):
...     def __init__(self, username):
...         super().__init__(username) [cite: 257]
...         self.setInt(10) [cite: 258]
...         self.setVit(5) [cite: 259]
...         self.setHp(self.getHp() + self.getVit()) [cite: 260]
...     def heal(self):
...         self.addHp(self.getInt()) [cite: 262]
...         print(f"{self.getUsername()} performed Heal! +{self.getInt()}") [cite: 263]
...     def magicAttack(self, character):
...         self.new_damage = self.getDamage() + self.getInt() [cite: 265]
...         character.reduceHp(self.new_damage) [cite: 266]
...         print(f"{self.getUsername()} performed Magic Attack! -{self.new_damage}") [cite: 267]
...
```

Code Analysis:

Code:

```
from Swordsman import Swordsman
from Archer import Archer
from Magician import Magician
from Boss import Boss

Character1 = Swordsman("Royce")
Character2 = Archer("Archie")
print(f"{Character1.getUsername()} - HP: {Character1.getHp()}, STR: {Character1.getStr()}, VIT: {Character1.getVit()}")
print(f"{Character2.getUsername()} - HP: {Character2.getHp()}, AGI: {Character2.getAgi()}, INT: {Character2.getInt()}")
Character1.slashAttack(Character2)
Character1.basicAttack(Character2)
print(f"{Character1.getUsername()} - HP: {Character2.getHp()}, AGI: {Character2.getAgi()}, INT: {Character2.getInt()}")
print(f"{Character2.getUsername()} - HP: {Character2.getHp()}, AGI: {Character2.getAgi()}, INT: {Character2.getInt()}")
Character2.heal()
Character2.basicAttack(Character1)
Character2.slashAttack(Character1)
Character2.rangedAttack(Character1)
Character2.magicAttack(Character1)
print(f"{Character1.getUsername()} - HP: {Character1.getHp()}, STR: {Character1.getStr()}, VIT: {Character1.getVit()}")
print(f"{Character2.getUsername()} - HP: {Character2.getHp()}, AGI: {Character2.getAgi()}, INT: {Character2.getInt()}")
```

Code:

Code Analysis:

Output

```
from Swordsman import Swordsman
from Archer import Archer
from Magician import Magician

class Boss(Swordsman, Archer, Magician):  # multiple inheritance
    def __init__(self, username):
        super().__init__(username)
        self.setStr(10)
        self.setVit(25)
        self.setInt(5)
        self.setHp(self.getHp() + self.getVit())
```

Code:

```
>>> class Boss(Swordsman, Archer, Magician):  # multiple inheritance
...     def __init__(self, username):
...         super().__init__(username)
...         self.setStr(10)
...         self.setVit(25)
...         self.setInt(5)
...         self.setHp(self.getHp() + self.getVit())
```

7. Supplementary Activity

8. Conclusion